보고된 정답률은 무엇이 결정하는가? LLM 기반 Triton GPU 커널 생성에 대한 실증 연구

Anonymous ACL submission

Abstract

최근 많은 논문이 대규모 언어 모델(LLM)로 GPU 커널을 작성하고, 동일한 벤치마크에서 정답률을 보고한다. 그러나 보고된 수치는 서로 크게 다르다. 어떤 논문은 거의 0에 가까운 값을, 다른 논문은 100%를 보고한다. 본 논문은 새로운 생성 시스템을 제안하지 않는다. 대신 이 격차가 무엇에서 비롯되는지를 묻는다. 상용 모델 8종과 오픈 모델 4종을 GPU 3종과 벤치마크 4종에서 측정하며, 모든 실행에 하나의 고정된 평가 하니스를 사용한다. 또한 Triton 특화 강화학습 모델, 에이전트 프레임워크, 실패 규칙 메모리, 프로파일링 피드백 등 기존 연구가 공개한 산출물을 가져와 본 연구의 환경에서 실행한다. 본 논문은 8가지 발견을 보고한다. 격차의 상당 부분은 모델에서 비롯되지 않는다. 그것은 평가 설정과 서빙 스택의 은닉된 한도에서 비롯된다. 이러한 결함 하나만으로도 정답률이 0.45에서 0.75로 움직인다. 나머지 격차의 대부분은 테스트 시점 연산 예산에서 비롯된다. 동일한 무료 8B 오픈 모델이 작은 예산에서는 0.44, 큰 예산에서는 0.95에 도달한다. 그러나 큰 예산은 학습 분포 내부에서만 도움이 된다. 분포 밖이나 새로운 도메인 특화 언어에서는 큰 예산이 아무것도 바꾸지 못한다. 또한 깊이보다 폭이 우세하며, 모델 크기보다 도메인 특화가 우세함을 확인한다. 마지막으로 논문이 정답률 수치와 함께 보고해야 할 6가지 축을 제시한다.

1 서론

GPU 커널은 딥러닝 시스템 비용의 상당 부분을 결정한다. 빠른 커널을 직접 작성하는 것은 어렵다. 프로그래머가 메모리 계층, 스레드 동기화, 워프 수준 동작을 모두 고려해야 하기 때문이다 (Kirk and Hwu, 2016). OpenAI Triton (Tillet et al., 2019)은 이 장벽을 낮춘다. Triton은 Python 기반의 타일 단위 프로그래밍 모델을 제공하고, 많은 저수준 선택을 컴파일러에 맡긴다. Triton은 PyTorch 2.0에서 `torch.compile`의 기본 백엔드가 되었으며 (Ansel et al., 2024), 따라서 이제 자동 코드 생성의 자연스러운 대상이다. 이에 많은 연구 그룹이 LLM으로 Triton 및 CUDA 커널을 생성하려 시도하고, 그 결과를 사실상 표준 벤치마크가 된 KernelBench (Ouyang et al., 2025)에서 측정한다.

문제는 보고된 수치가 서로 일치하지 않는다는 점이다. KernelEvolve는 100%를 보고한다 (Liao et al., 2025). CudaForge는 97.6%를 보고한다 (Zhang et al., 2025b). QiMeng은 처음 두 레벨에서 거의 완벽한 정답률을 보고한다 (Zhu et al., 2026). 동시에 같은 벤치마크에 대한 다른 연구들은 한 자릿수 정답률을 보고한다. 독자는 이 격차가 모델 능력에서 오는지, 탐색 예산에서 오는지, 아니면 측정 설정 자체에서 오는지 구분할 수 없다. 각 논문은 시도 횟수, 검증 강도, 서빙 스택이 모두 다르며, 대부분의 논문은 이러한 설정을 아예 보고하지 않는다.

따라서 본 논문은 시스템 논문이 아니다. 새로운 생성기, 새로운 에이전트, 새로운 학습 방법을 제안하지 않는다. 본 논문은 현재 분야의 상태를 감사(audit)한다. 하나의 평가 하니스를 고정하고 그 위에서 한 번에 하나의 축만 바꾼다. 모델, GPU, 벤치마크, 연산 예산을 변화시킨다. 또한 기존 연구가 공개한 산출물, 즉 Triton 특화 강화학습 모델 (Li et al., 2025b), 학습 환경을 포함한 다중 턴 강화학습 모델 (Liu et al., 2026), 오픈 딥 에이전트 프레임워크, 실패 규칙 메모리 (Du et al., 2026), 프로파일링 피드백 (Zhang et al., 2025b)을 가져온다. 각각을 본 연구의 환경에서 실행하고, 보고된 이점이 어디서 유지되고 어디서 깨지는지를 보고한다. 본 연구는 다섯 가지 질문을 중심으로 구성된다. RQ1은 하나의 공통 설정에서 현재 모델들이 실제로 얼마나 정확한지를 묻는다. RQ2는 측정 설정만으로 보고 수치가 얼마나 결정되는지를 묻는다. RQ3는 실패가 어떤 모습이며 모델이 발전함에 따라 어떻게 변하는지를 묻는다. RQ4는 수리의 깊이, 샘플링의 폭, 테스트 시점 예산, 강화학습 중 무엇이 실제로 정답률을 높이는지를 묻는다. RQ5는 기존 연구의 산출물이 다른 하니스와 다른 분포로 전이되는지를 묻는다.

본 연구는 100편 이상의 관련 논문을 CUDA 생성, Triton 생성, 강화학습, 에이전트 및 진화적

프레임워크, 컴파일러 및 IR 수준 연구, 벤치마크, 평가 방법론, 지도학습 기반 파운데이션 모델의 8개 그룹으로 정리하였다. 그 결과 두 가지 공백이 드러났다. 첫째, 많은 논문이 단일 기법을 제안하지만, 여러 모델, 여러 GPU, 여러 벤치마크를 하나의 통제된 하니스에서 비교한 논문은 찾지 못했다. 둘째, 완벽한 정답률을 보고하는 논문은 항상 무제한 트리 탐색, 프로파일링을 동반한 반복 수리, 대규모 강화학습과 같은 큰 시도 예산을 전제하지만, 단일 시도 수치를 함께 보고하지 않는다. 본 연구가 사용하는 반복 수리, best-of-N 샘플링, 모델 캐스케이딩, 실패 규칙 재사용 등 개별 구성 요소는 모두 알려진 것이다. 새로운 점은 그 위에서 수행한 통제된 측정이다.

본 연구의 기여는 새로운 방법이 아니라 일련의 발견이며, 이 절만으로도 연구 결과를 파악할 수 있도록 여기에 정리한다. 첫째, 최신 상용 플래그십 모델들은 일정 난이도 이하에서 거의 동일한 동작으로 수렴하며, 무료 8B Triton 특화 오픈 모델도 같은 수준에 도달한다. 즉 특화가 크기보다 우세하다. 둘째, 모델이 아닌 평가 인프라와 은닉된 서빙 한도만 바꿔도 정답률이 최대 0.30 움직이므로, 이러한 설정이 명시되지 않는 한 논문 간 정답률 수치는 비교할 수 없다. 셋째, 모델이 좋아질수록 지배적 실패 유형이 컴파일 에러에서 값 불일치로 이동하며, 이는 정답률이 해결되기 전에는 프로파일링 피드백이 도움이 되지 않는 이유도 설명한다. 넷째, 단일 샘플을 더 깊이 수리하는 것보다 독립 샘플이 더 효과적이다. 다섯째, 본 연구의 전체 벤치마크 수치와 문헌상의 완벽한 점수 사이의 격차는 대부분 예산 체제의 차이이며, 강화학습과 예산은 대체재가 아니라 보완재이다. 여섯째, 에이전트 프레임워크는 탐색 효율을 높이지만 기반 모델의 상한을 끌어올리지는 못하므로, 강력한 오픈 모델과 오픈 프레임워크의 조합만으로도 상용 모델 없이 완벽한 점수에 도달할 수 있다. 일곱째, 공격적인 양자화는 어려운 과제에서 품질을 무너뜨리고, 초대형 모델의 전정밀도 실행은 처리량을 무너뜨리므로, 전정밀도의 중간 크기 모델이 실용적인 선택이다. 여덟째, 그리고 전체 그림의 한계로서 가장 중요한 점은, 큰 예산이 학습 분포 내부에서만 도움이 된다는 것이다. 분포 밖 과제와 새로운 도메인 특화 언어는 전혀 개선되지 않는다. 이러한 발견으로부터 향후 논문이 정답률과 함께 보고해야 할 6가지 축을 도출한다.

2 관련 연구

벤치마크와 학습. KernelBench (Ouyang et al., 2025)는 3개 레벨에 걸쳐 250개 과제를 정의하고, 모델에게 PyTorch 레퍼런스를 동등한 GPU 커널로 변환하도록 요구한다. TritonBench (Li et al., 2025a)는 GitHub에서 실제 Triton 연산자를 수집하며, TritonGym (Guan et al., 2026)은 도구 접근을 표준화하여 모델 능력과 워크플로 설계를 분리해 측정할 수 있게 한다. 빠르게 변화하는 이 분야를 정리한 여러 서베이가 있다 (Yu et al., 2026; Zhang et al., 2026; Gong et al., 2025). 학습 측면에서 AutoTriton (Li et al., 2025b)은 지도학습과 GRPO를 결합하여 8B 모델로 상용 수준의 품질을 보고한다. TritonRL (Woo et al., 2025)은 검증 가능한 보상으로 보상 해킹을 억제하고, DRTriton (Guo et al., 2026)은 대규모 합성 데이터를 사용한다. CUDA의 경우 CUDA-L1 (Li et al., 2025c)은 대조 강화학습을, Kevin (Baronio et al., 2025)은 다중 턴 강화학습을 사용한다. Tehrani et al. (2026)은 프런티어 모델을 미세조정하여 단일 시도 정답률을 높인다. Dr. Kernel (Liu et al., 2026)은 커널 강화학습의 두 가지 실패 양상, 즉 보상 해킹과 게으른 최적화(lazy optimization)를 지적하고, 분산 학습 환경과 다중 턴 어드밴티지 추정기를 함께 공개한다. CUDA Agent (Dai et al., 2026)와 MusaCoder (Cheng et al., 2026)는 이 흐름을 대규모 에이전트 학습과 새로운 가속기로 확장한다. 이러한 방법들은 모두 데이터 큐레이션과 막대한 연산을 필요로 한다. 본 연구는 아무것도 학습하지 않는다. 이들 논문이 공개한 것을 내려받아 본 연구의 하니스에서 실행하고, 보고된 이점이 하니스 변경과 분포 변경 후에도 유지되는지를 묻는다.

에이전트 프레임워크. 두 번째 흐름은 고정된 모델을 수리 루프 안에서 사용하며 학습을 추가하지 않는다. GEAK (Wang et al., 2025)은 생성기, 리플렉터, 평가기, 최적화기를 사용한다. CudaForge (Zhang et al., 2025b)는 하드웨어 프로파일링 지표를 추가한다. EGG (Han et al., 2026)는 커널 생성을 알고리즘 설계와 하드웨어 튜닝으로 분리하고, KernelBand (Ran et al., 2026)는 최적화를 밴딧 문제로 다룬다. STARK (Dong et al., 2025)는 계획, 코딩, 디버깅을 분리하고, PRAGMA (Lei et al., 2025)는 프로파일링 데이터를 명시적 추론 단계로 변환한다. AccelOpt (Zhang et al., 2025a)는 경험 메모리를 재사용하고, KernelEvolve (Liao et al., 2025)는 여러 하드웨어 플랫폼에 걸쳐 트리 탐색을 수행한다. AdaExplore (Du et al., 2026)는 반복되는 실패를 재사용 가능한 규칙으로 증류하고 탐색의 다양성을 유지한다. KForge (Sereda et al., 2026)와 Kernel Foundry (Huang et al., 2026)는 정답률 단계와 최적화 단계를 번갈아 수행한다. 다른 연구들은 원시 CUDA 대신 간결한 언어를 노출하여 추론 부담을 줄이거나(μCUTLASS (Hari et al., 2026),

CuTeGen (Saba et al., 2026), QiMeng-Kernel (Zhu et al., 2026)), 하드웨어 맥락을 추가한다(SwizzlePerf (Tschand et al., 2025)). LEGO (Xu et al., 2026)는 단일 커널에서 전체 모델 그래프로 범위를 넓힌다. 이러한 프레임워크는 모두 큰 시도 예산을 전제하며, 이것이 바로 본 연구가 통제하는 축이다.

검증과 평가. 생성된 커널은 컴파일되더라도 틀릴 수 있으므로, 검증은 이제 독립된 연구 주제가 되었다. Kwon et al. (2026)는 컴파일러와 런타임 사이에 검증 체인을 구축한다. COLT (Tang et al., 2026)는 여러 소형 모델이 협력하다가 필요할 때 더 큰 모델로 승격시키며, Crupi et al. (2026)는 소형 모델을 심사자로 학습시켜 올바른 후보를 저비용으로 선택한다. AutoMegaKernel (Jaber and Jaber, 2026)은 정적 검사기로 잘못된 제안을 실행 전에 거부하고, AgentCompile (Li et al., 2026)은 정답 여부 판정을 컴파일러 내부에 둔다. 검증되지 않은 수치의 위험은 실재한다. 보상 해킹으로 부풀려진 결과가 이미 보고된 바 있기 때문이다 (Lange et al., 2025). 본 연구는 이 우려를 검증에서 측정 설정 전체로 확장한다.

3 연구 설계

그림 1는 연구의 구조를 보여준다. 모든 실행은 하나의 하니스를 공유한다. 하니스는 프롬프트를 구성하고, 모델을 호출하고, 응답에서 코드를 추출하고, 격리된 서브프로세스 안에서 커널을 채점하고, 에러를 분류한 뒤, 수리하고 재시도한다. 격리 단계는 최적화가 아니다. CUDA 런타임은 프로세스 단위로 상태를 유지하므로, 잘못된 커널 하나가 컨텍스트를 오염시켜 같은 프로세스에서 이후의 모든 평가를 실패하게 만들 수 있다. 격리 없이는 250개 과제에 대한 순차 실행이 의미를 갖지 못한다. 정답률 판정을 위해 무작위 입력 두 개를 생성하고, 두 입력 모두에서 레퍼런스 대비 절대 오차가 10^{-2} 미만일 것을 요구한다. 속도는 10회 측정 실행의 중앙값을 취하며, 벤치마크의 정의 (Ouyang et al., 2025)에 따라 Fast-0, Fast-1, Fast-2를 보고한다.

이 고정된 하니스 위에서 네 가지 축을 변화시킨다. 첫 번째 축은 모델이다. 상용 모델 8종, 즉 gpt-4o-mini, gpt-4o, gpt-5.2, gpt-5.2-pro, gpt-5.4-nano, gpt-5.5, gemini-3.1-pro-preview, claude-opus-4-7을 사용한다. 또한 오픈 모델 4종, 즉 Qwen3-Coder-30B-A3B (Hui et al., 2024), AutoTriton-8B (Li et al., 2025b), 2비트 및 4비트로 양자화한 GLM-5.2, Dr. Kernel-14B (Liu et al., 2026)를 사용한다. 두 번째 축은 하드웨어이다. 96GB RTX PRO 6000, 80GB H100, 48GB A6000을 사용한다. 세 번째 축은 벤치마크이다.

Table 1: 작은 예산에서 KernelBench Level 1 전체 100개 과제의 정답률. 모든 행은 동일한 하니스와 동일한 과제 집합을 사용한다. AutoTriton 행은 독립 실행 4회의 평균과 표준편차이며, 나머지 행은 모두 단일 실행 결과이다.

모델	유형	Fast-0
gpt-4o-mini	상용	15%
gpt-4o	상용	19%
Qwen3-Coder-30B	오픈, 30B	27%
gpt-5.4-nano	상용	35%
gpt-5.5	상용	37%
gemini-3.1-pro-preview	상용	43%
claude-opus-4-7	상용 플래그십	46%
AutoTriton-8B	오픈, 8B, RL	$44\% \pm 4$

250개 과제의 KernelBench, 166개 과제의 TritonBench, 표준 139개 과제와 분포 밖 13개 과제의 TritonGym, 그리고 8개 dense 과제의 FlashInfer-Bench (FlashInfer Team, 2025)를 사용한다. 네 번째 축은 연산 예산이다. 샘플 수, 수리 라운드 수, 다중 턴 설정에서의 유효 턴 수를 1부터 14까지 변화시킨다. 상용 모델은 통합 게이트웨이를 통해 실행한다. 오픈 모델은 로컬 vLLM 서버에서 실행하며, 양자화 모델은 llama.cpp에서 실행한다.

소프트웨어 스택은 H100에서 Triton 3.5.1, RTX PRO 6000에서 Triton 3.6이며, PyTorch 2.x, Python 3.12, CUDA 12.x를 사용한다. 샘플 1개와 최대 3회의 수리 라운드를 사용하는 경우를 *작은* 예산, 샘플 3개와 최대 9회의 수리 라운드, 즉 과제당 최대 27회 시도를 사용하는 경우를 *큰* 예산이라 부른다.

4 실험 결과

4.1 RQ1: 현재 모델은 얼마나 정확한가?

먼저 작은 예산을 고정하고 모든 모델을 KernelBench Level 1의 100개 과제 전체에 대해 실행한다. 표 1에 결과를 보고하며, 과제의 일부만 실행한 경우는 제외하였다.

모델 세대의 효과는 뚜렷하다. 정답률은 gpt-4o의 19%에서 최신 플래그십 모델의 43–46%까지 증가한다. 더 흥미로운 것은 마지막 행이다. Triton에 특화되어 학습된, 파라미터 8B에 불과한 무료 오픈 모델이 최고 수준의 상용 플래그십과 같은 수준에 도달한다. 동일한 하니스와 예산으로 이 실행을 4회 반복한 결과 0.47, 0.44, 0.39, 0.47을 얻었으므로, 이 벤치마크의 단일 실행은 약 4포인트의 표준편차를 갖는다. 8B 모델과 최고 플래그십 사이의 2포인트 차이처럼 이 정도 크기의 차이는 동률로 보아야 하며, 모델 세대 간 격차는 이보다 한 자릿수 더 크다. 4회의 실행은

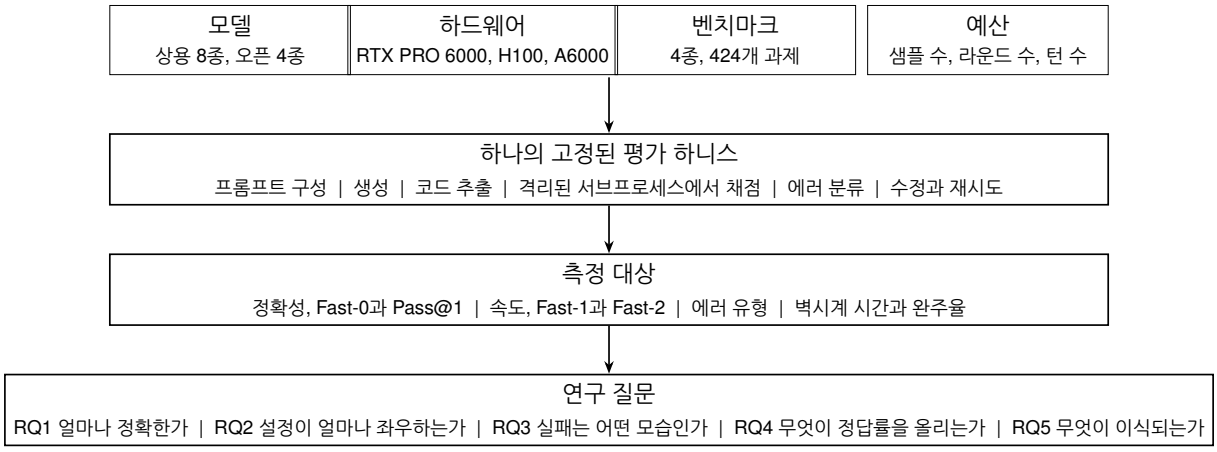


Figure 1: 본 연구의 구조. 하나의 평가 하니스를 고정하고 그 위에서 네 개의 축을 바꾼다. 하니스는 모든 커널을 격리된 서브프로세스에서 채점한다. 이는 선택이 아니라 필수인데, 잘못된 커널 하나가 CUDA 컨텍스트를 오염시키면 같은 프로세스의 이후 평가가 모두 실패하기 때문이다. 우리는 새로운 생성 시스템을 제안하지 않는다. 하니스를 고정한 채 한 번에 하나의 축만 바꾸어 모델, 하드웨어, 벤치마크, 계산 예산의 효과를 분리한다.

서로 다른 59개 과제를 해결했지만 공통으로 해결한 과제는 25개뿐이었다(Finding 4 참조). 또한 gpt-5.5와 claude-opus-4-7은 처음 44개 과제 중 정확히 같은 37개를 해결하고 벤치마크의 마지막 부분에서야 갈라지며, gemini-3.1-pro-preview도 같은 후반부 회복을 보이므로, 이는 특정 모델이 아니라 플래그십 세대의 특성으로 보인다.

Finding 1. *최근 상용 플래그십 모델들은 일정 난이도 이하에서 거의 동일한 동작으로 수렴하며, 무료 8B Triton 특화 오픈 모델도 같은 수준에 도달한다. 커널 생성에서는 도메인 특화가 모델 크기보다 우세하다.*

15–46%라는 이 절대 수치는 문헌상의 완벽한 점수에 크게 못 미친다. 다음 두 절은 이 격차의 대부분이 모델 능력이 아닌 다른 축에서 비롯됨을 보인다.

4.2 RQ2: 설정은 얼마나 결정하는가?

본 연구의 파이프라인을 감사하는 과정에서 모델 능력과 무관하게 정답률을 떨어뜨리는 일련의 결함을 발견하였다. 표 2는 이들 결함과 수정 후의 회복을 함께 정리한다.

첫 번째 행이 가장 명확한 사례이다. 서빙 프로세스가 GPU의 약 90%를 점유하면, 채점기는 입력이 큰 과제에서 메모리 부족을 일으킨다. Level 1의 ReLU 과제 하나는 단일 입력 텐서가 6.4GB이다. 그 결과 의미상 자명한 ReLU조차 하니스가 실패로 기록하였다. 서빙 점유율을 낮추고 매 시행 후 텐서를 해제하자 모델을 전혀 바꾸지 않고도 정답률이 0.45에서 0.75로 상승하였다. 나머지 행들도 같은 성격을 갖는다. 하니스는 이들을 모델이 풀지 못한 과제로 집계하지만, 실제로는 코드가 도착하기 전에 잘렸거나 채점기가 스스로 고장 난 것이었다.

이 중 세 가지는 은닉 변수이다. 프레임워크의 응답 토큰 한도, 공개된 모델 산출물 내부의 생성 한도, 서빙 컨텍스트 길이는 어떤 정답률 보고에도 등장하지 않는다. 추론 모델은 코드를 쓰기 전에 사고 과정에 수천 토큰을 소비하므로, 흔히 쓰이는 8K 한도는 풀 수 있는 과제를 빈 응답으로 바꾸어 버린다.

Finding 2. *모델은 건드리지 않고 평가 인프라와 은닉된 서빙 한도만 바꿔도 정답률이 최대 0.30 움직인다. 따라서 이러한 설정이 보고되지 않는 한 서로 다른 시스템의 정답률 수치는 비교할 수 없다.*

본 연구의 코드베이스도 두 번째 사례를 제공한다. 모델, 프롬프트, 예산은 같지만 스레드 수, 타임아웃, temperature, 검증 입력이 다른 두 평가 경로는 같은 8B 모델에 대해 각각 0.44 ± 0.04와 0.35 ± 0.05를 기록한다(부록 M).

또한 각 실패를 하나의 짧은 규칙으로 증류하여 과제 간에 재사용하는 기존 연구의 실패 규칙 메모리 (Du et al., 2026)를 이식하였다. 메모리 결함이 여전히 존재하던 시기에 시스템이 수집한 규칙들은 잘못된 것이었다. 예를 들어 GPU에 커널을 실행할 메모리가 충분하지 않다는 규칙이 있었다. 이 잘못된 규칙들은 높은 가중치를 가지고 프롬프트를 채웠으며, 그 결과 모델은 단순한 정사각 행렬 곱셈에서조차 실패하였다. 저장소를 초기화한 뒤 정답률은 회복되었다. 따라서 경험 재사용은 평가 환경이 이미 건전할 때에만 자산이 되며, 공유 저장소는 그 피해를 여러 서버로 확산시킨다.

4.3 RQ3: 실패는 어떤 모습인가?

각 실패를 컴파일 에러, 값 불일치, CUDA 에러, 타임아웃 중 하나로 분류한다. Table 3에 그 분포

Table 2: 측정 설정의 결함과 수정 후의 회복. 모든 행에서 모델은 동일하며 환경만 바뀐다.

결함	증상	수정	정답률
GPU 메모리 경합	서빙이 GPU의 90%를 점유하여 큰 입력이 메모리 부족(OOM)으로 실패	서빙 점유율을 낮추고 시행마다 텐서를 해제	0.45 → 0.75
입력 dtype 계약 부재	레퍼런스는 fp32인데 모델은 fp16을 가정	프롬프트에 dtype 계약을 명시	0.65 → 1.00
게이팅 및 컨텍스트 초기화 부재	오염된 컨텍스트가 모든 수리 라운드를 가로막음	정답률로 게이팅하고 깨끗한 컨텍스트에서 재시작	
프레임워크 토큰 한도	코드 블록 이전에 추론 과정이 잘림	응답 토큰 한도를 상향	0.37 → 0.78
산출물 생성 한도	서빙 측 한도가 클라이언트 한도를 덮어씀	산출물의 생성 설정을 수정	잘림 해소
서빙 컨텍스트 길이	수리 프롬프트가 넘쳐 모든 라운드가 소실됨	서빙 컨텍스트를 확장	실패 해소

Table 3: KernelBench Level 1에서 실패한 과제 중 각 에러 유형이 차지하는 비율. 타임아웃 열은 본 연구가 설정한 응답 및 평가 시간 한도와 함께 API 요청 한도 같은 미분류 런타임 에러를 합산한 것이다. 벤치마크 자체의 제약이 아니다.

모델	컴파일	값 불일치	CUDA	타임아웃
gpt-4o-mini	71.8%	20.0%	2.4%	2.4%
gpt-4o	45.7%	37.0%	11.1%	0%
gpt-5.2	6.9%	58.6%	1.7%	20.7%
claude-opus-4-7	0%	65%	0%	28%

를 제시한다.

모델이 향상될수록 지배적 에러 유형은 규칙적으로 이동한다. 컴파일 에러는 71.8%에서 0%로 감소하는 반면, 값 불일치는 20%에서 65%로 증가한다. 언어 모델이 Triton의 문법을 학습하는 것은 비교적 쉽지만, GPU 연산의 수치적 의미를 이해하는 것은 여전히 어렵다.

이 진단은 실용적 귀결을 가진다. 세 모델을 대상으로 최적화 프롬프트에 Nsight Compute 하드웨어 카운터를 주입하는 A/B 테스트를 수행하였다. 어느 모델에서도 정답률은 개선되지 않았고, 속도 향상 이득도 매우 작았다. 프로파일링 신호는 이미 정답인 커널에만 적용되는데, 실제 병목은 정답률이다. 따라서 프로파일링 기반 최적화는 정답률 문제가 해결된 이후에야 의미가 있다.

Finding 3. *모델이 향상될수록 실패는 문법적 에러에서 의미적 에러로 이동한다. 정답률이 해결되기 전에는 프로파일링 피드백이 효과가 없으므로, 정답률을 먼저 확보하고 프로파일링은 나중에 적용하는 설계가 필요하다.*

타임아웃 열은 별도로 해석해야 한다. 추론 모델인 gpt-5.2-pro의 경우 Level 1 실패의 100%가 타임아웃이었다. 긴 사고 과정이 본 연구의 응답 시간 한도 안에 들어가지 않았으므로, 이는 능력의 한계가 아니라 실험 설정의 운영상 제약이다.

Table 4: KernelBench Level 1에서 gpt-4o의 수리 라운드 수에 따른 효과.

라운드	해결	컴파일 에러	값 불일치
1	19	59.3%	24.7%
3	21	51.9%	39.2%
5	19	45.7%	37.0%
10	22	32.1%	53.8%
합집합	**28**	—	—

4.4 RQ4: 무엇이 실제로 정답률을 높이는가?

4.4.1 깊이 대 너비.

gpt-4o를 Level 1에서 수리 라운드 1, 3, 5, 10회로 실행하였다. 개별 실행은 각각 19, 21, 19, 22개 과제를 해결하여, 더 깊은 수리만으로는 거의 도움이 되지 않았다. 그러나 네 실행의 합집합은 28개 과제를 해결하여 최선의 단일 실행보다 27.3% 높았다. 즉 각 실행은 다른 실행이 놓친 과제를 일부 해결한다. Section 4.1의 동일 조건 AutoTriton 4회 실행도 예산 변경 없이 같은 효과를 보인다. 그 합집합은 59개 과제를 포괄하는 반면 최선의 실행은 47개이다. Table 4는 에러 프로필이 깊이에 따라 이동함도 보여준다. 라운드가 늘어날수록 컴파일 에러는 제거되지만 값 불일치는 남으며, 이는 Finding 3과 일치한다.

같은 효과는 모델 간에도 성립한다. 캐스케이드에서 gpt-4o 단독으로는 25개 과제를 해결하였고, 다음 세대 모델을 추가하자 총 61개로 늘어 2.44배가 되었다. 다섯 모델의 합집합은 100개 중 51개에 도달하여, 100개 중 46개인 최선의 단일 모델을 10.9% 앞섰다. 이득은 균일하지 않았다. 마지막 단계에 고비용 추론 모델을 추가하자 225개 중 단 1개 과제만 늘어 그 비용을 정당화하지 못했다. 또한 최근의 플래그십 모델 하나만으로도 100개 중 46개를 달성하여 이전의 3-모델 캐스케이드(100개 중 43개)를 이미 능가하므로, 더 나은 모델 세대는 다중 모델 전략의 필요성을 줄

Table 5: 예산만 변경했을 때의 Level 1 정답률. 작은 예산은 샘플 1개와 라운드 3회이다. 큰 예산은 샘플 3개와 라운드 9회이다. 단검표로 표시한 값은 시간 한도 안에 완료된 38개 과제를 기준으로 계산하였다. AT-8B의 작은 예산 값은 4회 실행의 평균이며(Section 4.1), 나머지 값은 모두 단일 실행이다.

예산	범위	Qwen-30B	AT-8B	opus
작음	전체 100	0.27	0.44	—
큼	처음 20	0.75	**0.95**	**1.00**
큼	전체 100	—	0.76†	—

Table 6: 유효 턴 수에 따른 Fast@1.2(%). 턴 수만 변경한다. DrK-14B는 커널 특화 강화학습으로 학습되었고 Qwen3-30B는 그렇지 않다.

모델 및 레벨	1턴	3턴	7턴	14턴
Qwen3-30B, L2	3.7	11.9	16.8	28.4
DrK-14B, L2	17.4	41.3	60.6	**76.9**
DrK-14B, L1	11.5	22.0	27.0	31.2
DrK-14B, L3	3.0	8.3	13.0	20.5

인다.

Finding 4. *너비가 깊이를 이긴다. 같은 예산이라면 하나의 시도를 더 깊이 수리하는 것보다 여러 번의 독립적 시도가 더 유용하다.*

4.4.2 테스트 시점 예산.

예산 축만 바꾸면 격차의 대부분이 사라진다. Table 5에서 무료 8B 모델은 작은 예산의 0.44에서 큰 예산의 0.95로 상승하며, 이는 문헌의 완벽한 점수에 근접한다.

그러나 더 큰 예산은 벽시계 시간에 의해 제한된다. 큰 예산의 전체 실행에서는 과제당 27회의 시도가 느린 로컬 서버의 2시간 한도 안에 들어가지 않아 과제의 59%가 완료되지 못했다. 따라서 테스트 시점 스케일링의 실제 병목은 시도 횟수가 아니라 시도당 처리량이다. 이는 완벽한 정답률을 보고하는 논문들이 빠른 상용 API나 대규모 서빙 클러스터를 전제하는 이유도 설명한다.

4.4.3 강화학습과 예산.

공개된 다중 턴 강화학습 환경을 재현하고 유효 턴 수만 1에서 14까지 변경하였다. Table 6는 이러한 학습을 거치지 않은 모델과 거친 모델을 비교한다.

두 모델 모두 예산에 따라 단조 증가하므로, 예산 지배는 특정 모델의 성질이 아니라 일반적 효과이다. 그러나 학습된 모델은 절편(1턴에서 4.7배)과 턴당 이득(59.5포인트 대 24.7포인트)을 모두 높인다. 포화 지점 또한 난이도에 따라 달라진다. Level 1은 약 7턴에서 포화되는 반면, Level 3은 14턴에서도 포화되지 않으며 가장 큰 상대적 이득을 보인다.

Finding 5. *문헌의 완벽한 점수와 본 연구의 전체 벤치마크 수치는 서로 다른 모델 능력이 아니라 서로 다른 예산 체제를 측정한 것이다. 강화학습과 테스트 시점 예산은 상호 보완적이며, 포화 지점은 과제 난이도에 따라 달라진다.*

4.5 RQ5: 다른 설정으로 무엇이 전이되는가?

4.5.1 실패 규칙 메모리의 재사용.

Section 4.2의 오염 사례와 별개로, 실패 규칙 메모리는 하니스가 정상일 때 유용하다. 시스템은 각 실패를 하나의 짧은 규칙으로 변환하는데, 예를 들어 Triton 커널 본문 안에서 PyTorch 연산을 호출해서는 안 된다는 규칙이다. 이러한 규칙은 과제 간에 유지된다. 한 번의 전체 평가 동안 저장소에는 73개에서 80개의 서로 다른 규칙이 축적되었으며, 이 규칙들은 잘못된 타일 차원, 리덕션 축을 따라 잘못된 블록 크기, 연속성(contiguity) 가정 누락 같은 흔한 실수를 포괄하였다. 이런 메모리는 실패당 모델 호출 1회만 추가로 필요하므로 비용이 낮으며, 본 연구가 재사용한 모든 산출물 중 이식성이 가장 높다. 그러나 그 위험은 비대칭적이다. 유용한 규칙은 몇 번의 시도를 절약하는 데 그치지만, 높은 가중치를 가진 잘못된 규칙은 과제 유형 전체를 막을 수 있다. 정사각 행렬 곱셈처럼 단순한 과제에서조차 정답률이 붕괴한 것이 그 예이다.

4.5.2 공개된 학습 환경의 재현.

먼저 커널 특화 강화학습 연구의 공식 코드와 공개 가중치를 내려받아 보고된 수치가 재현되는지 확인하였다. Level 2의 100개 과제에서 Fast@1.2는 24.3%로 측정되어 논문의 25.6%와 비교되었고, Fast@1.5와 Fast@2는 오히려 더 높았다. 따라서 보고된 결과는 실재한다. 그러나 재현에는 8-GPU 노드가 필요했다. 단일 GPU에서는 트레이너와 서빙 엔진이 함께 적재되지 않았으므로, 재현 가능성이 접근성을 의미하지는 않는다.

4.5.3 하드웨어 간 및 벤치마크 간 결과.

Table 7는 두 오픈 모델을 두 GPU에서 모든 레벨에 걸쳐 보고한다. 8B Triton 특화 모델의 우위는 일관되며, Level 2와 Level 3에서 난이도 절벽이 가파르다. 정답률은 단일 연산의 0.44에서 융합 연산의 0.10, 전체 모델의 0.06으로 떨어진다. FlashInfer-Bench의 실제 서빙 커널에서는 한 조합만 0.25에 도달하고 나머지는 0에 머무는데, 이는 Triton 버전과 연산 아키텍처의 차이를 시사한다.

메모리가 충분한 한 정답률 자체는 GPU와 거의 무관하다. 처음 20개 과제에 대해 같은 큰 예

Table 7: 작은 예산에서 두 GPU에 걸친 오픈 모델의 전 레벨 결과. PRO 6000은 RTX PRO 6000이며 AT-8B는 Triton 특화 모델이다. H100에서는 30B 모델이 채점기에 너무 적은 메모리만 남기므로, 단검표로 표시한 값은 메모리 부족(OOM) 실패로 낮아진 값이다. Finding 2를 참조하라. PRO 6000의 Level 1 AT-8B 값은 4회 실행의 평균이며, 나머지 값은 모두 단일 실행이다.

레벨	Qwen-30B		AT-8B	
	PRO 6000	H100	PRO 6000	H100
L1, 100개 과제	0.27	0.12†	**0.44**	0.43
L2, 100개 과제	0.05	0.06	**0.10**	0.08
L3, 50개 과제	0.04	0.06	**0.06**	0.02
FlashInfer, 8	0.00	0.00	0.00	**0.25**

산을 주었을 때 30B 오픈 모델은 RTX PRO 6000에서 0.75, H100에서 0.60에 도달하였으며, 그 차이는 전적으로 메모리에서 비롯되었다. H100은 80GB이므로 30B 모델을 서빙한 뒤 채점기에는 약 43GB만 남았지만, 96GB인 RTX PRO 6000에서는 약 52GB가 남았다. 메모리 부족 실패를 보정하면 H100의 수치는 약 0.70으로 상승한다. 세 번째 GPU인 48GB의 A6000도 Level 1에서 0.37, Level 2에서 0.12, Level 3에서 0.04로 레벨 전반에 걸쳐 유사한 프로필을 보여 같은 결론을 뒷받침한다. 다만 A6000의 처리량은 H100의 약 4분의 1에 불과하여, 같은 스윕에 벽시계 시간이 약 3배 더 걸렸다.

속도 향상은 반대로 동작한다. 정사각 행렬 곱셈에서 RTX PRO 6000은 1.61배, H100은 0.83배를 기록하였는데, 이는 H100의 cuBLAS 기준선이 매우 강력하기 때문이다. 따라서 속도 향상 수치는 하나의 하드웨어 설정 안에서만 비교 가능하다. 최적화의 질 또한 모델 세대에 따라 향상된다. 정답 커널 중 2배 이상의 속도 향상에 도달한 비율은 gpt-4o-mini의 6.3%, gpt-4o의 16.0%에서 gpt-5.4-nano의 28.6%, claude-opus-4-7의 30.4%로 증가한다. 관측된 최대 단일 속도 향상은 Level 3의 한 과제에서 84.03배였다.

4.5.4 에이전트 구조 대 모델 능력.

완벽한 정답률을 보고하는 오픈 딥 에이전트 프레임워크를 내려받아 본 연구의 오픈 모델로 실행하였다. Qwen-30B로는 처음 20개 과제 중 15개를 해결하였는데, 이는 큰 예산에서 본 연구의 파이프라인이 달성한 점수와 정확히 같으며 실패한 과제 집합까지 동일하였다. 그러나 벽시계 시간은 수 시간에서 약 1시간으로 줄었다. 더 강력한 오픈 모델인 4-bit GLM-5.2로는 같은 프레임워크가 20개 과제를 모두 해결하여 상용 플래그십 모델과 대등하였다.

Finding 6. *에이전트 프레임워크는 탐색 효율을 높이지만 기반 모델의 상한을 끌어올리지는 않는다. 정답률은 프레임워크 효율과 모델 능력의 곱이며, 강력한 오픈 모델과 오픈 프레임워크의 조합은 상용 모델 없이도 완벽한 점수에 도달할 수 있다.*

이 조합을 Level 1 전체로 확장하자 실행한 77개 과제에서 0.48을 얻었다. 성공은 행렬 곱셈, 리덕션, 원소별 연산 과제에 집중되었고, 합성곱과 순차 스캔 과제는 급격히 떨어졌다. 예산과 능력을 분리하기 위해 32K 서빙 컨텍스트와 24000 토큰의 응답 한도로 잘림을 완전히 제거하고, 대표적인 합성곱 과제 3개를 재시도하였다. 잘림은 사라졌지만 세 과제 모두 여전히 수렴하지 못했다. 따라서 남은 합성곱 실패는 토큰 예산의 한계가 아니라 대형 텐서 인덱싱에 대한 모델 능력의 한계이다.

4.5.5 모델 크기 대 가중치 정밀도.

매우 큰 오픈 추론 모델인 GLM-5.2를 8-GPU 노드에 레이어 분할로 적재하였다. 2-bit 형태에서는 21.5시간의 연산 끝에 Level 1에서 0.10에 그쳤고, Level 2, Level 3, 서빙 커널에서는 단 한 과제도 해결하지 못했다. 즉 파라미터 수가 두 자릿수 배 많은 일반 추론 모델이 Table 7의 8B 특화 모델의 절반에도 미치지 못했다. 양자화가 원인임을 확인하기 위해 같은 모델을 4-bit로 다시 적재하였다. 시간 한도 안에 완료된 과제 기준 정답률은 0.96으로 회복되어 오픈 모델 중 최고 결과를 보였다. 그러나 이 버전은 초당 약 8토큰만 생성하여 과제의 76%가 한도를 초과하였다. 병목이 정밀도에서 처리량으로 옮겨갔을 뿐이다.

Finding 7. *양자화 손실은 과제 복잡도에 따라 커진다. 낮은 정밀도는 품질을, 초대형 모델의 온전한 정밀도는 처리량을 무너뜨리므로, 고정된 예산에서는 온전한 정밀도의 중형 모델이 실용적인 선택이다.*

4.5.6 예산의 경계.

위의 모든 결과는 학습 분포 안에서 성립한다. 그 경계를 찾기 위해 커널 특화 강화학습 모델을 표준화된 에이전트 환경으로 옮겨 단일 시도와 테스트 시점 스케일링 에이전트를 비교하였다. 139개 표준 과제에서 Pass@1은 12.2%에서 18.0%로 상승하였고, 통과한 커널의 오라클 대비 성능은 0.143에서 0.285로 상승하였다. 그러나 13개 분포 외(OOD) 과제에서 Pass@1은 7.7%에 머물러 전혀 움직이지 않았다. 새로운 도메인 특화 언어에서는 같은 모델이 본 연구의 환경에서 실행 가능한 3개 과제 중 하나도 해결하지 못했다. 부록 J는 같은 시험을 두 NPU 타깃에서 반복한다. 같은 패턴이 벤치마크 간에도 나타난다. Triton 특화 벤치마크에서는 특화 모델의 우위가 사라지고 네 조합 모두 0.15에서 0.19 사이의 좁은 범위로 수

렴한다.

원인을 분리하기 위해 프롬프트 형식과 예산에 대한 2×2 ablation을 수행하였다. 프롬프트 형식만 바꾸는 것은 거의 효과가 없었으며, 실행 정확도는 오히려 0.151에서 0.127로 떨어졌다. 반면 예산을 켜자 실행 정확도는 약 2.3배, 호출 정확도는 약 2.5배 상승하였다. 따라서 원 논문과의 격차는 학습 프롬프트와의 불일치가 아니라 단일 시도 평가 대 반복 수리의 차이로 설명된다. 이는 KernelBench에서 발견한 바를 반복하므로, 예산 지배는 특정 벤치마크의 성질이 아니라 일반적 효과이다.

> **Finding 8.** *더 큰 테스트 시점 예산은 분포 안에서만 정답률과 성능을 향상시킨다. 분포 밖의 붕괴는 구제하지 못한다. 강화학습의 이득 또한 자체 학습 하니스에 묶여 있어 부분적으로만 전이된다.*

5 결론

본 논문은 새로운 시스템을 제안하는 대신 다섯 가지 연구 질문으로 LLM 기반 Triton 커널 생성의 현황을 감사한다. 하나의 고정된 하니스로 상용 모델 8개와 오픈 모델 4개를 세 GPU와 네 벤치마크에서 측정하였고, 기존 연구의 공개 산출물을 본 연구의 설정에서 실행하였다. 시스템의 보고된 정답률은 모델 능력만의 함수가 아니다. 그것은 능력, 테스트 시점 예산, 측정 설정의 함수이다. 인프라 결함을 제거하고, 충분한 예산을 주고, 특화된 오픈 모델을 사용하는 것만으로 상용 모델 없이도 보고된 최고 결과 수준에 도달할 수 있다. 그러나 이 처방은 학습 분포 안에서만 작동하며, 분포 밖에서는 어떤 예산도 도움이 되지 않는다. 따라서 향후 연구는 정답률 수치와 함께 다음 여섯 축을 보고할 것을 요청한다: 시도 예산, 검증 강도, 채점기의 메모리 여유, 서빙 컨텍스트 길이, 프레임워크와 모델 산출물 양쪽의 응답 토큰 상한, 평가 분포.

Limitations

여러 번 실행한 구성은 Level 1에서 작은 예산의 8B 오픈 모델 하나뿐이다. 그 실행 간 표준편차는 약 4포인트이며(Section 4.1), 본 논문의 다른 모든 수치는 단일 실행에서 얻었으므로 표의 행 간 몇 포인트 차이를 순위로 해석해서는 안 된다. 또한 Section 4.2의 두 평가 경로가 보여주듯, 절댓값은 본 연구가 명시한 하니스 설정에 묶여 있다. 부록 J의 두 NPU 실험은 각각 연산자 5개와 커널 3개만을 다루며, 두 번째 실험은 오픈 모델 하나로만 수행되었으므로 벤치마크가 아니라 패턴의 증거로 읽어야 한다. 큰 예산 측정 일부는 Level 1의 100개 과제 전체가 아니라 처음 20개 과제에서 수행되었고, 전체 실행에서는 벽시계 한도 때문에 완료되지 못한 과제가 남았다. 상용 모델은 API 뒤에 있어 서빙 조건을 통제할 수 없으므로, 상용 모델과 오픈 모델 간의 모든 비교에는 통제되지 않은 인프라 차이가 남아 있다. 분포 외(OOD) 집합은 13개 과제뿐이므로 경계의 존재는 보여주지만 그 크기를 정밀하게 측정하지는 못한다. 마지막으로 타임아웃 열은 실제 시간 한도와 미분류 런타임 에러를 섞고 있으므로, 단일 실패 원인이 아니라 운영상의 범주로 읽어야 한다.

Ethical Considerations

본 연구는 인간 피험자, 개인 데이터, 주석자를 포함하지 않는다. 본 연구가 재사용한 모든 벤치마크와 모델 산출물은 저자들이 연구 목적으로 공개한 것이며, 각각을 인용하였다. KernelBench 코드, TritonGym, AdaExplore, GLM-5.2 가중치는 MIT 라이선스로, TritonBench, FlashInferBench, Qwen3-Coder 가중치, AutoTriton 가중치는 Apache-2.0으로 공개되어 있으며, Dr. Kernel 학습 코드는 문서에 Apache-2.0을 명시한다. KernelBench 과제 데이터와 Dr. Kernel 가중치는 명시적 라이선스가 없으며, 저자들의 의도대로 평가 목적으로만 사용한다. 부록 J의 두 NPU 소프트웨어 스택은 본 연구가 사용할 수 있는 하드웨어에서 각 벤더의 약관에 따라 사용한 벤더 SDK이다. 어떤 가중치나 데이터도 재배포하지 않으며, 본 제출과 함께 새로운 산출물을 공개하지 않는다. 본 연구의 주된 위험은 생성된 커널을 검증 없이 프로덕션에 오용하는 것이며, 검증 강도를 여섯 가지 보고 축 중 하나로 삼음으로써 이를 다룬다. 연산 비용을 발견과 비교해 따져볼 수 있도록 각 실험의 벽시계 비용을 Appendix B에 보고한다.

References

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. 2024. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *ASPLOS*.

Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. 2025. Kevin: Multi-turn rl for generating cuda kernels. *Preprint*, arXiv:2507.11948.

Kun Cheng, Songshuo Lu, Sicong Liao, Tankun Li, Yafei Zhang, et al. 2026. MusaCoder: Native GPU kernel generation with full-stack training on Moore Threads GPUs. *Preprint*, arXiv:2606.04847.

Giuseppe Crupi, Rosalia Tufano, and Gabriele Bavota. 2026. Improving code generation via small language model-as-a-judge. In *Proceedings of the IEEE/ACM*

48th International Conference on Software Engineering (ICSE).

Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, et al. 2026. CUDA Agent: Large-scale agentic reinforcement learning for high-performance CUDA kernel generation. *Preprint*, arXiv:2602.24286.

Juncheng Dong, Yang Yang, Tao Liu, Yang Wang, Feng Qi, Vahid Tarokh, Kaushik Rangadurai, and Shuang Yang. 2025. Stark: Strategic team of agents for refining kernels. *Preprint*, arXiv:2510.16996.

Weihua Du, Jingming Zhuo, Yixin Dong, Andre Wang He, Weiwei Sun, Zeyu Zheng, Manupa Karunaratne, Ivan Fox, Tim Dettmers, Tianqi Chen, Yiming Yang, and Sean Welleck. 2026. Adaexplore: Failure-driven adaptation and diversity-preserving search for efficient kernel generation. *Preprint*, arXiv:2604.16625.

FlashInfer Team. 2025. FlashInfer-Bench: Building the virtuous cycle for AI-driven LLM systems. https://github.com/flashinfer-ai/flashinfer-bench. Benchmark suite of production LLM serving kernels; blogpost at https://flashinfer.ai/2025/10/21/flashinfer-bench.html.

Jingzhi Gong, Vardan Voskanyan, Paul Brookes, Fan Wu, Wei Jie, Jie Xu, Rafail Giavrimis, Mike Basios, Leslie Kanthan, and Zheng Wang. 2025. Language models for code optimization: Survey, challenges and future directions. *Preprint*, arXiv:2501.01277.

Yue Guan, Yichen Lin, Xu Zhao, Jianzhu Yao, Xinwei Qiang, Zhongkai Yu, Pramod Viswanath, Yufei Ding, and Adnan Aziz. 2026. TritonGym: A benchmark for agentic LLM workflows in Triton GPU code generation. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. Code and leaderboard: https://github.com/yil384/TritonGym.

Siqi Guo, Ming Lin, and Tianbao Yang. 2026. Dr-triton: Large-scale synthetic data driven reinforcement learning for triton kernel generation. *Preprint*, arXiv:2603.21465.

Yaochen Han, Ke Fan, Hongxu Jiang, Wanqi Xu, Weiyu Xie, Runhua Zhang, Chenhui Zhu, and Yixiang Zhang. 2026. EGG: An expert-guided agent framework for kernel generation. *Preprint*, arXiv:2606.26758.

Siva Kumar Sastry Hari, Vignesh Balaji, Sana Damani, Qijing Huang, and Christos Kozyrakis. 2026. Improving efficiency of GPU kernel optimization agents using a domain-specific language and speed-of-light guidance. *Preprint*, arXiv:2603.29010.

Zixuan Huang, Da Chen, Kecheng Huang, Lihao Yin, Xing Li, Huiling Zhen, Mingxuan Yuan, and Zili Shao. 2026. Kernel Foundry: A diagnosis-driven evolutionary kernel optimizer with multi-experts. *Preprint*, arXiv:2605.30359.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*.

Jaber Jaber and Osama Jaber. 2026. AutoMegaKernel: A statically-checked agent harness for self-retargeting megakernel synthesis. *Preprint*, arXiv:2606.09682.

David B. Kirk and Wen-mei W. Hwu. 2016. *Programming Massively Parallel Processors: A Hands-on Approach*, 3rd edition. Morgan Kaufmann.

Hyunho Kwon, Sanggyu Shin, Ju Min Lee, Hoyun Youm, Seungbin Song, Seongho Kim, Hanwoong Jung, Seungwon Lee, and Hanjun Kim. 2026. Compiler-runtime co-operative chain of verification for LLM-based code optimization. In *Proceedings of the 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*.

Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. 2025. The AI CUDA engineer: Agentic CUDA kernel discovery, optimization and composition. Technical report, Sakana AI.

Kelun Lei, Hailong Yang, Huaitao Zhang, Xin You, Kaige Zhang, Zhongzhi Luan, Yi Liu, and Depei Qian. 2025. Pragma: A profiling-reasoned multi-agent framework for automatic kernel optimization. *Preprint*, arXiv:2511.06345.

Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025a. Tritonbench: Benchmarking large language model capabilities for generating triton operators. In *Findings of the Association for Computational Linguistics (ACL)*.

Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025b. AutoTriton: Automatic triton programming with reinforcement learning in LLMs. *arXiv preprint arXiv:2507.05687*.

Xiaoya Li, Albert Wang, Guoyin Wang, Jiwei Li, and Chris Shum. 2025c. Cuda-l1: Improving cuda optimization via contrastive reinforcement learning. *Preprint*, arXiv:2507.14111.

Xuanzhe Li, Ziyan Weng, Zhiyu Zhu, and Junhui Hou. 2026. AgentCompile: An LLM-guided compiler for direct CUDA inference. *Preprint*, arXiv:2606.07665.

Gang Liao, Carole-Jean Wu, Gaoxiang Liu, et al. 2025. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta. *Preprint*, arXiv:2512.23236.

Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. 2026. Dr. Kernel: Reinforcement learning done right for Triton kernel generations. In *Proceedings of the 43rd International*

Conference on Machine Learning (ICML). KernelGYM environment and code available at https://github.com/hkust-nlp/KernelGYM.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. ArXiv:2502.10517.

Dezhi Ran, Shuxiao Xie, Mingfang Ji, Anmin Liu, Mengzhou Wu, Yuan Cao, Yuzhe Guo, Hao Yu, Linyi Li, Yitao Hu, Wei Yang, and Tao Xie. 2026. KernelBand: Steering LLM-based kernel optimization via hardware-aware multi-armed bandits. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. Code available at https://github.com/TongmingLAIC/KernelBand.

Tara Saba, Zhiyang Chen, Jikai Jason Li, Anne Ouyang, Xujie Si, and Fan Long. 2026. Cutegen: An llm-based agentic framework for generation and optimization of high-performance gpu kernels using cute. *Preprint*, arXiv:2604.01489.

Taras Sereda, Burak Bartan, Ankita Nayak, Tom St. John, Natalie Serrino, and Zain Asgar. 2026. KForge: LLM-driven cross-platform kernel generation for AI accelerators. In *Machine Learning for Architecture and Systems Workshop (MLArchSys), co-located with ISCA*.

Annabelle Sujun Tang, Christopher Priebe, Lianhui Qin, and Hadi Esmaeilzadeh. 2026. COLT: Lightweight multi-LLM collaboration through shared MCTS reasoning for model compilation. *Preprint*, arXiv:2602.01935.

Ali Tehrani, Yahya Emara, Essam Wissam, Wojciech Paluch, Waleed Atallah, Łukasz Dudziak, and Mohamed S. Abdelfattah. 2026. Fine-tuning gpt-5 for gpu kernel generation. *Preprint*, arXiv:2602.11000.

Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An intermediate language and compiler for tiled neural network computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19.

Arya Tschand, Muhammad Awad, Ryan Swann, Kesavan Ramakrishnan, Jeffrey Ma, Keith Lowery, Ganesh Dasika, and Vijay Janapa Reddi. 2025. SwizzlePerf: Hardware-aware LLMs for GPU kernel performance optimization. *Preprint*, arXiv:2508.20258.

Jianghui Wang, Vinay Joshi, Saptarshi Majumder, Xu Chao, Bin Ding, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. 2025. Geak: Introducing triton kernel ai agent & evaluation benchmarks. *Preprint*, arXiv:2507.23194.

Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. 2025. TritonRL: Training LLMs to think and code triton without cheating. *arXiv preprint arXiv:2510.17891*.

Ruiyuan Xu, Shuoming Zhang, Guangli Li, Qiuchu Yu, Rui Zhang, Yangyu Zhang, Hao Qian, Chunwei Xia, Jiacheng Zhao, Chenxi Wang, Xiaobing Feng, Jingling Xue, and Huimin Cui. 2026. LEGO: An LLM-enabled hierarchical optimizer for tensor computation graphs with structure-aware search and compositional synthesis. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. Code available at https://github.com/mmt-at/LEGO.

Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, Guang Liu, and Yonghua Lin. 2026. Towards automated kernel generation in the era of LLMs. *arXiv preprint arXiv:2601.15727*.

Genghan Zhang, Shaowei Zhu, Anjiang Wei, Zhenyu Song, Allen Nie, Zhen Jia, Nandita Vijaykumar, Yida Wang, and Kunle Olukotun. 2025a. Accelopt: A self-improving llm agentic system for ai accelerator kernel optimization. *Preprint*, arXiv:2511.15915.

Shuoming Zhang, Jiacheng Zhao, Qiuchu Yu, Chunwei Xia, Zheng Wang, Xiaobing Feng, and Huimin Cui. 2026. The new compiler stack: A survey on the synergy of LLMs and compilers. *Preprint*, arXiv:2601.02045.

Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. 2025b. Cudaforge: An agent framework with hardware feedback for cuda kernel optimization. *Preprint*, arXiv:2511.01884.

Xinguo Zhu, Shaohui Peng, Jiaming Guo, Yunji Chen, Qi Guo, Yuanbo Wen, Hang Qin, Ruizhi Chen, Qirui Zhou, Ke Gao, Yanjun Wu, Chen Zhao, and Ling Li. 2026. QiMeng-Kernel: Macro-thinking micro-coding paradigm for LLM-based high-performance GPU kernel generation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.

A 하니스 구현

A.1 격리된 채점

모든 커널은 각자의 하위 프로세스에서 평가한다. 하위 프로세스는 생성된 모듈을 로드하고, 정답 검사와 시간 측정 루프를 실행한 뒤, 결과를 표준 출력에 한 줄의 JSON으로 기록한다. 부모 프로세스는 그 줄을 읽어 결과를 수집한다. 이 설계는 선택이 아니라 필수이다. CUDA 런타임은 프로세스 단위로 상태를 유지하므로, 잘못된 커널 하나가 컨텍스트를 손상시키면 같은 프로세스에서 이후에 수행되는 모든 평가도 함께 실패한다. 격리 없이 250개 과제를 순차 실행하는 것은 의미가 없다.

A.2 정답 검사와 청크 비교

두 개의 무작위 입력을 생성하고, 두 입력 모두에서 레퍼런스 대비 절대 오차가 10^{-2} 미만이면 커

널을 정답으로 인정한다(프레임워크의 기본 경로는 다섯 개를 생성한다. 부록 M 참조). 단순한 비교는 절대 차이, 허용 오차, 마스크를 위한 임시 텐서를 할당한다. 이들은 각각 출력과 같은 크기를 가지므로, 최대 메모리는 출력 크기의 세 배에서 네 배까지 증가한다. 48GB GPU에서는 이 임시 텐서만으로도 *올바른* 커널이 메모리 부족으로 실패하였다. 이를 피하기 위해 6400만 원소 단위의 청크로 비교하며, 비교 과정에서 진단 통계도 함께 수집한다. 입력 텐서는 순전파 직후 해제하고, 메모리 단편화를 줄이는 할당자 옵션으로 워커를 실행한다. 속도는 10회 실행의 중앙값을 사용하며, 속도 향상은 레퍼런스 시간을 생성 커널 시간으로 나눈 값이다.

A.3 프롬프트 구조

시스템 프롬프트는 대상 GPU의 하드웨어 사양을 명시한다. 약 100KB, 즉 101,376바이트인 공유 메모리 크기, 디바이스 메모리 용량, 메모리 대역폭, 최대 처리량을 제공한다. 또한 간단한 타일 크기 규칙, 예를 들어 행렬 곱셈에는 64 이하의 블록 크기, 원소별 연산에는 1024의 블록 크기를 제시한다. 여기에 소규모 Triton 패턴 데이터베이스와 아키텍처 힌트 집합도 주입한다. 사용자 프롬프트는 전체 PyTorch 레퍼런스와 **입력 dtype 계약**을 포함한다. 이 계약은 모델이 레퍼런스 입력의 dtype을 추측하거나 형 변환해서는 안 된다는 것을 알려 준다. 이 계약이 없으면 모델은 fp16을 가정한 커널을 작성하고, 그러한 커널은 모두 fp32 레퍼런스에서 실패한다. 수리 프롬프트에는 이전 코드, 에러 메시지, 현재 라운드 번호가 추가된다. 응답은 `torch.nn.Module`을 상속한 `Model` 클래스와 `@triton.jit` 데코레이터가 붙은 커널 함수를 포함하는 하나의 Python 코드 블록이어야 한다.

A.4 수리 루프의 안전 규칙

루프는 커널이 정답일 때에만 최적화 단계로 진입한다. 값이 일치하지 않으면 커널을 빠르게 만들려고 시도하지 않는다. 정해진 라운드 수 동안 진전이 없으면 누적된 컨텍스트와 이전 코드를 폐기하고 처음부터 다시 생성한다. 이 두 번째 규칙은 예를 들어 불법 메모리 접근 이후처럼 컨텍스트가 오염된 경우에는 도움이 되지만, 모델이 과제를 단순히 풀지 못하는 경우에는 도움이 되지 않는다. 실험에서 두 경우는 쉽게 구분되었다. 컨텍스트 초기화로 구제된 과제는 두 번의 초기화 후에도 실패한 과제와 달랐으며, 후자는 3차원 및 4차원 텐서 곱과 전치 행렬 곱셈에 집중되어 있었다.

A.5 응답 파싱과 다중 API

파서는 Markdown 코드 블록을 추출하고, 클래스 이름을 `Model`로 바꾸며, Triton 코드가 아닌 설명문을 제거한다. 이 단계가 모델마다 다른 출력 형식을 흡수한다. 상용 모델은 chat completion API를 사용한다. 추론 모델은 temperature 파라미터를 받지 않으므로 대신 response API를 사용한다. 오픈 모델은 로컬 vLLM 서버의 completion API를 사용하고, 양자화 모델은 llama.cpp를 사용한다. 클라이언트는 모델 식별자로부터 적절한 API를 판별하고, 첫 호출이 실패하면 대체 경로로 전환하며, 매 호출마다 같은 판별을 반복하지 않도록 결과를 캐시한다.

B 예산 설정과 벽시계 시간 비용

소규모 예산은 샘플 1개와 최대 3회의 수리 라운드이다. 이 예산은 250개 과제 전체 스윕에 사용한다. 대규모 예산은 샘플 3개와 최대 9회의 수리 라운드로, 과제당 최대 27회 시도이다. 멀티턴 곡선은 유효 턴 수만 1에서 14까지 변화시킨다.

대규모 예산에서의 전체 스윕은 벽시계 시간 비용이 지배한다. 느린 로컬 서버에서는 과제당 27회 시도가 2시간 제한 안에 들어가지 않았다. 그 결과 과제의 59퍼센트가 완료되지 못했고, 전체 스윕에 22.9시간이 걸렸다. 결과를 분해하면, 정상 평가 후 정답 29개, 오답 9개로 완료된 과제는 38개뿐이었다. 나머지는 워커 타임아웃 59건과 결과 없음 3건이었다. 주목할 점이 하나 있다. 여러 과제가 처음에는 27회 시도를 모두 실패한 과제처럼 보였으나, 실제로는 전부 타임아웃이었고 오답이 아니었다. 표면 정확도만 보는 독자는 이를 모델 능력의 한계로 오해하기 쉽다.

여기서 세 가지 설계 규칙을 도출한다. 첫째, 워커 타임아웃은 고정된 추정치가 아니라 시도 횟수와 시도당 평균 시간의 곱으로 설정한다. 둘째, 워커 수는 동시 서빙 슬롯 수보다 적게 유지한다. 셋째, best-of-N의 샘플은 순차적이 아니라 병렬로 생성한다.

과제 수준의 병렬화는 해로웠다. 두 과제를 동시에 실행하자 서빙 슬롯이 포화되고, 슬롯당 처리량이 절반 이하로 떨어졌으며, 생성이 클라이언트 타임아웃을 초과하고, 재시도 루프가 끝없이 반복되어 어떤 과제도 완료되지 못했다. 본 설정에서는 순차 실행이 더 나은 선택이었다.

부록 M의 반복 실행은 모델 서빙과 커널 채점을 동시에 수행한 RTX PRO 6000 한 대에서 회당 4.3–4.9시간, 총 약 32 GPU-시간이 걸렸다. 이 GPU에서 소규모 예산의 Level 1 전체 스윕은 매번 이 비용이 들며, Level 2와 Level 3 스윕은 레퍼런스 모델이 더 크기 때문에 더 오래 걸린다.

C 수리 라운드 절제 실험의 원자료

표 8은 본문 표 4의 근거가 되는 전체 수치를 제시한다.

Table 8: 수리 라운드 수가 정확도와 에러 분포에 미치는 영향. 모델은 gpt-4o, 과제 집합은 KernelBench Level 1이다.

라운드	해결 (/100)	컴파일 에러	값 불일치
1	19	59.3%	24.7%
3	21	51.9%	39.2%
5	19	45.7%	37.0%
10	22	32.1%	53.8%
합집합	**28**	—	—

라운드 수가 증가함에 따라 컴파일 에러의 비중은 59.3퍼센트에서 32.1퍼센트로 감소하는 반면, 값 불일치의 비중은 24.7퍼센트에서 53.8퍼센트로 증가한다. 반복 수리는 구문 오류는 잘 고치지만 의미적 오류는 스스로 고치지 못한다. 이는 본문의 에러 전이 발견과 일치한다. 실행 간 합집합은 19, 25, 26, 28로 증가하여, 22개 과제를 해결한 최고 단일 실행보다 항상 높게 유지된다.

D 모델 캐스케이딩 상세

본문은 캐스케이드 실험의 결론만 보고한다. 표 9은 각 단계의 누적 결과를 제시한다.

Table 9: 캐스케이드의 누적 정확도. 각 단계는 이전 단계가 실패한 과제에 대해서만 실행된다.

구성	L1	L2	L3	합계 (/250)
gpt-4o-mini	15	1	0	16 (6.4%)
gpt-4o	19	6	0	25 (10.0%)
gpt-4o + gpt-5.2	42	15	4	61 (24.4%)
+ gpt-5.2-pro	43	15	4	62 (24.8%)
claude-opus-4-7 단독	46	–	–	–
opus + gpt-5.4-nano	48	–	–	–
5개 모델 합집합	**51**	–	–	–

세 가지를 관찰한다. 첫째, gpt-4o에 차세대 모델을 추가하면 합계가 25개에서 61개로, 2.44배 증가한다. 새로 해결된 36개 과제는 Level 1 23개, Level 2 9개, Level 3 4개이다. 둘째, 고가의 추론 모델은 거의 아무것도 추가하지 못한다. 이 모델은 225개 과제에서 실행되어 단 1개, 속도 향상 7.60배의 Level 1 과제만을 추가하였다. 비용과 응답 시간까지 고려하면 캐스케이드 내에서 이 모델의 가치는 0에 가깝다. 셋째, 최신 플래그십 모델 하나가 단독으로 46개를 해결하여 이전의 3개 모델 캐스케이드(43개)를 이미 능가한다. 5개 모델 합집합(51개)은 여전히 최고 단일 모델보다 10.9퍼센트 높으므로, 독립 샘플의 가치는 최신 세대에서도 사라지지 않는다.

E Level별 에러 분포

본문의 표 3은 Level 1만 다룬다. 표 10은 세 Level 모두를 다룬다. 타임아웃 열은 자체 응답 및 평가 시간 제한과 분류되지 않은 런타임 에러를 합산한 것이다. 이는 벤치마크의 제약이 아니다.

Table 10: 모델별, Level별 실패 과제 중 각 에러 유형의 비중.

모델	Lv.	실패	컴파일	값	CUDA	타임아웃
gpt-4o-mini	L1	85	71.8%	20.0%	2.4%	2.4%
	L2	99	70.7%	22.2%	4.0%	1.0%
	L3	50	82.0%	6.0%	6.0%	0%
gpt-4o	L1	81	45.7%	37.0%	11.1%	0%
	L2	94	51.1%	41.5%	4.3%	0%
	L3	50	38.0%	56.0%	4.0%	0%
gpt-5.2	L1	58	6.9%	58.6%	1.7%	20.7%
	L2	85	25.9%	62.4%	0%	8.2%
	L3	46	17.4%	71.7%	0%	4.3%
gpt-5.5	L1	63	71%	3%	0%	25%
claude-opus-4-7	L1	54	0%	65%	0%	28%

에러 전이는 모든 Level에서 나타난다. gpt-4o-mini는 세 Level 모두에서 컴파일 에러가 지배적이고, gpt-5.2는 세 Level 모두에서 값 불일치가 지배적이다. 한 모델 내에서도 난이도에 따라 같은 이동이 나타난다. gpt-4o의 값 불일치 비중은 Level 1의 37.0퍼센트에서 Level 3의 56.0퍼센트로 증가하므로, 과제가 어려워질수록 실패는 더 의미적인 성격을 띤다.

추론 모델 gpt-5.2-pro의 경우 Level 1 실패의 100퍼센트, Level 2 실패의 81.9퍼센트가 타임아웃이었다. Level 3에서는 모든 실패가 API 속도 제한(rate limit)에서 비롯되었다. 이는 다른 원인이므로 타임아웃 열에 합산하지 않는다.

F 속도 향상 결과 상세

속도 향상은 정답 검사를 통과한 커널에 대해서만 측정한다. 표 11에 결과를 보고한다.

Table 11: 정답 커널 중 Fast-1 및 Fast-2 임계값에 도달한 비중. 분모는 해당 모델의 정답 커널 수이다.

모델	Fast-1	Fast-2
gpt-4o-mini	7/16 (43.8%)	1/16 (6.3%)
gpt-4o	12/25 (48.0%)	4/25 (16.0%)
gpt-5.2	25/36 (69.4%)	5/36 (13.9%)
gpt-5.2-pro	10/13 (76.9%)	3/13 (23.1%)
gpt-5.4-nano	21/35 (60.0%)	10/35 (28.6%)
gpt-5.5	22/37 (59.5%)	8/37 (21.6%)
claude-opus-4-7	29/46 (63.0%)	14/46 (30.4%)

Fast-2 비중은 모델 세대에 걸쳐 6.3퍼센트에서 30.4퍼센트로 증가하므로, 최신 모델은 더 많은 정답 커널뿐 아니라 더 잘 최적화된 커널도 작성한다. 추론 모델은 흥미로운 예외이다. 이 모델은 표에서 가장 적은 13개 과제만 해결했지만, Fast-2 비중은 23.1퍼센트로 높다. 추론 모델은 과제를 해결할 때에는 잘 최적화된 커널을 작성하는 경향이 있다. 그러나 전체 성공률이 낮으므로 실용적 가치는 제한적이다.

관찰된 최대 단일 속도 향상은 Level 3 과제 하나에서의 84.03배였다. 그 밖의 높은 사례는 25.45배, 19.10배, 11.86배였다. 이 사례들은 생성된 Triton 커널이 일부 연산 패턴에서 PyTorch 기본 구현을 큰 차이로 능가할 수 있음을 보여 준다. 그러나 속도 향상은 하드웨어 기준선에 의존하므로, 이 수치는 동일한 하드웨어 설정 내에서만 비교 가능하다.

G 재사용한 공개 산출물 상세

본문은 각 이식(port)의 결과만 보고한다. 여기서는 각 산출물의 구성 방식과 실행 시 발생한 일을 기술한다.

G.1 실패 규칙 메모리

이 시스템은 각 실패를 "이것을 하면 안 된다" 형태의 짧은 규칙 하나로 변환하고, 규칙을 점수와 함께 저장하며, 정규화 후 중복을 제거하고, 상위 규칙을 생성 프롬프트에 주입한다. 한 번의 전체 평가에서 저장소에는 73–80개의 서로 다른 규칙이 수집되었다. 예로는 커널 본문에서 PyTorch 연산을 호출해서는 안 된다는 규칙, 행렬 곱셈의 차원과 축소 축 방향 블록 크기에 관한 규칙, 연속성(contiguity) 가정에 관한 규칙 등이 있다. 비용은 실패당 추가 모델 호출 1회로 저렴하며, 재사용한 산출물 중 이식성이 가장 높았다.

그러나 위험은 비대칭적이다. 좋은 규칙은 몇 번의 시도를 절약하지만, 높은 점수를 가진 잘못된 규칙은 과제 부류 전체를 막을 수 있다. 실험에서 평가 하니스가 아직 고장 나 있던 동안 수집된 규칙이 저장소를 오염시켰다. 오염된 저장소에는 320개의 규칙이 있었다. 그중 일부는 인프라 결함을 모델에 대한 규칙으로 취급하였는데, 예를 들어 GPU에 커널을 실행할 메모리가 부족하다는 규칙이 있었다. 이 규칙들은 점수가 높아 프롬프트 상단을 차지하였다. 그 결과 모델은 9회의 수리 라운드와 3개의 샘플을 사용하고도 정사각 행렬 곱셈처럼 단순한 과제에서조차 실패하였다. 저장소를 초기화한 후 정확도가 회복되었다. 교훈은 두 가지이다. 인프라를 수정한 후에는 반드시 저장소를 초기화해야 하며, 저장소를 공유하는 다중 서버 환경에서는 오염이 한 서버에서 다른 서버로 전파된다.

G.2 Triton 특화 강화학습 모델

공개된 가중치를 로컬 vLLM 서버에서 서빙하였다. 이 모델은 채팅 형식의 system 역할을 지원하지 않아 초기 호출이 실패하였다. system 메시지를 사용자 프롬프트에 병합하도록 클라이언트를 수정하자 호출이 정상 동작하였다. Level 1의 처음 20개 과제에서 이 모델은 96GB GPU에서 0.85, H100에서 0.95를 기록하였다. H100 결과가 더 높은 이유는 모델이 약 16GB만 필요하여 채점기에 55GB 이상이 남았고, 메모리 부족으로 실패한 과제가 없었기 때문이다. 동일 조건에서 30B 범용 오픈 모델은 H100에서 0.60에 그쳤다. 따라서 소형 특화 모델은 대형 범용 모델을 능가하는 동시에 채점기에 더 많은 메모리를 남기며, 이는 실무적으로 중요하다.

G.3 오픈 딥 에이전트 프레임워크

공개된 프레임워크를 복제하여 자체 오픈 모델로 실행하였다. 이식 과정에서 세 가지 문제가 발생하였다. 첫째, 서빙 컨텍스트 길이가 수리의 작동 여부 자체를 결정한다. 16K 컨텍스트에서는 코드와 에러 텍스트에 따라 커지는 수리 프롬프트가 입출력 한도를 초과하여 모든 라운드가 소실되었다. 64K 컨텍스트에서는 같은 과제가 수십 초 만에 해결되었다. 둘째, 외부 프로젝트를 공유 환경에 설치하면 의존성이 깨진다. 프레임워크의 editable 설치가 공유 패키지 버전을 올려 서빙 엔진이 시작되지 않았다. 외부 프로젝트는 별도의 가상 환경에 설치해야 한다. 셋째, HTTP 응답이 에러일 때에도 셸 명령이 종료 코드 0을 반환하여, 서버가 정상이 아닌데도 정상이라고 두 번 잘못 판단하였다. 상태 확인은 실제 completion 호출의 상태 코드를 확인해야 한다.

또한 이 프레임워크는 모델 자신이 작성한 테스트로 답을 검증하며, 허용 오차가 본 연구보다 느슨하다. 따라서 그 정확도 수치와 본 연구의 정확도 수치는 서로 다른 검증 강도를 사용하며, 이는 서로 다른 시스템의 수치를 직접 비교해서는 안 되는 또 하나의 이유이다.

G.4 멀티턴 강화학습 환경

공개된 환경을 Level 2의 100개 과제에서 재현하였다. 표 12는 본문에 나열되지 않은 지표를 제시한다.

Fast@1.2는 거의 같고 Fast@1.5와 Fast@2는 더 높으므로, 재현에 성공한 것으로 판단한다. Fast@1은 6포인트 낮은데, 이는 GPU 차이와 샘플링 잡음에 기인한 것으로 본다. 정답률만으로는 76.3퍼센트였고, 느슨한 임계값에서의 pass@1은 85.7퍼센트였다.

Table 12: 공개된 멀티턴 강화학습 환경의 KernelBench Level 2(100개 과제) 재현 결과. 모든 값은 전체 과제에 대해 계산하였다.

지표	원 논문	본 재현
Fast@1	49.2%	42.9%
Fast@1.2	25.6%	24.3%
Fast@1.5	7.4%	12.9%
Fast@2	2.1%	7.5%

인프라 비용은 높았다. 트레이너의 채점 경로는 FSDP actor와 vLLM rollout 엔진을 같은 메모리에 올려야 하며, 96GB GPU 한 대에서의 다섯 번의 시도는 모두 실패하였다. 실질적으로 8-GPU 노드가 최소 요구 사항이었다. 또한 다른 버전의 잔여 프로세스가 작업에 붙는 문제 때문에 매 실행 전 기존 클러스터를 완전히 정리해야 했다. 공개된 검증 데이터는 Level 2만 포함하므로, Level 1과 Level 3에서 같은 재현을 하려면 동일 형식의 새 데이터셋이 필요하다. 이 절의 요점은 재현 가능성이 접근 용이성을 의미하지는 않는다는 것이다.

H 양자화 연구 상세

초대형 오픈 추론 모델을 공격적 양자화로 로드하여 동일한 프로토콜로 평가하였다. 레이어 분할 파이프라인 병렬화로 서빙하였고, 채점용으로 별도의 GPU 한 대를 유지하였다.

2비트 버전은 약 254GB였다. Level 1에서 0.10을 기록하였고, Level 2, Level 3, 실제 서빙 커널에서는 하나도 해결하지 못하였다. Level당 약 8.5시간, 총 21.5시간이 소요되었다. 시도당 약 2,600토큰을 사고(thinking)에 소비했으나 정확도 이득은 없었다. 따라서 매개변수가 두 자릿수 배 더 많은 범용 추론 모델이 8B 특화 모델의 절반에도 미치지 못하였다.

4비트 버전은 약 467GB로, 서빙 예산에 들어가는 최고 정밀도였다. 시간 제한 내에 완료된 과제에 대해서는 정확도가 0.96으로 회복되어, 양자화 손실이 붕괴의 원인임을 확인하였다. 그러나 이 버전은 초당 약 8토큰만 생성하였다. 그 결과 과제의 76퍼센트가 2시간 제한을 초과하였고, 표면 정확도는 0.23에 불과하였으며, 실행에 30시간이 걸렸다.

종합하면, 고정된 GPU 예산 하에서 초대형 모델은 저정밀도에서는 품질을 잃고, 같은 모델을 완전 정밀도로 쓰면 속도를 잃는다. 커널 생성에는 완전 정밀도의 중형 모델이 더 나은 선택이다.

I 교차 벤치마크 연구 상세

I.1 Triton 특화 벤치마크

이 벤치마크는 166개 과제로 구성되며 KernelBench와 다른 평가 방식을 사용한다. 각 과제는 PyTorch 함수와 테스트 하니스로 이루어지므로, 모듈과 입력 생성기를 기대하는 채점기로는 실행할 수 없다. 따라서 별도의 러너를 작성하였다. 러너는 레퍼런스와 생성 코드를 같은 테스트 하니스로 실행하고, 두 결과 딕셔너리를 비교하여 코드의 실행 여부와 결과 일치 여부를 모두 보고한다. 두 모델과 두 GPU의 네 조합에서 실행 정확도는 0.151–0.187의 좁은 범위에 머물렀으며, KernelBench에서 보였던 특화 모델의 우위는 사라졌다. 이 스위트에는 옵티마이저 업데이트처럼 Triton 커널로 자연스럽게 대응되지 않는 과제도 포함되어 있어, 스위트의 일부는 구조적으로 도달 불가능하다.

원 논문과의 격차 원인을 찾기 위해 프롬프트 형식과 예산에 대한 2요인(각 2수준) 절제 실험을 수행하였다. 프롬프트를 특화 모델의 학습 형식에 맞추는 것만으로는 도움이 되지 않았으며, 실행 정확도는 오히려 0.151에서 0.127로 떨어졌다. 예산을 적용하자 실행 정확도는 약 2.3배, 호출 정확도는 약 2.5배 증가하였다. 더 큰 예산 하에서는 PyTorch 레퍼런스를 제공하는 변환형 프롬프트가 정렬된 프롬프트보다 오히려 우수하였다. 따라서 격차는 학습 프롬프트와의 불일치가 아니라 단일 시도 평가로 설명된다.

I.2 표준화된 에이전트 환경

이 환경은 표준 과제 139개와 분포 외(OOD) 과제 13개로 구성된다. 단일 시도에서 139개 표준 과제 중 106개가 컴파일되었고, 17개(12.2퍼센트)가 컴파일과 정답 검사를 모두 통과하였다. 통과한 커널의 오라클 대비 성능 기하평균은 0.143으로 오라클보다 약 7배 느렸으며, 오라클보다 빠른 커널은 3개뿐이었다. 컴파일 실패의 주원인은 import 에러로, 모델이 존재하지 않는 모듈이나 심볼을 사용한 경우가 29건이었다. 13개 분포 외 과제에서는 12개가 컴파일되었지만 통과한 것은 1개(7.7퍼센트)뿐이었다.

테스트 시점 스케일링 에이전트를 사용하면 표준 과제의 Pass@1은 18.0퍼센트로, 성능 기하평균은 0.285로 약 두 배 상승하였다. 그러나 수리루프의 마지막 제안이 컴파일되지 않는 경우가 있어 컴파일률은 45.3퍼센트로 떨어졌다. 분포 외 과제에서 Pass@1은 7.7퍼센트에 머물며 전혀 움직이지 않았다. 이것이 본문의 경계 결과를 뒷받침하는 측정이다.

I.3 새로운 도메인 특화 언어 트랙

같은 환경에는 새로운 도메인 특화 언어를 위한 12개 과제 트랙이 있다. 이 중 9개 과제는 본 환경에서 아예 실행할 수 없었다. 일부는 별도의 컴파일러 포크가 필요하고, 다른 일부는 최신 버전에만 존재하는 API를 사용한다. 오라클이 실행 가능한 3개 과제에서 특화 강화학습 모델은 단일 시도로 하나도 통과하지 못하였다. 표본이 작으므로 정량적 결론은 내리지 않는다. 학습 분포 밖에서의 붕괴가 한 벤치마크의 한 축에 국한되지 않는다는 증거로서 보고한다.

I.4 실제 서빙 커널 벤치마크

최신 NVIDIA 아키텍처용 실제 서빙 커널 벤치마크에서 밀집(dense) 과제 8개를 가져와 KernelBench 형식으로 변환하였다. 변환은 레퍼런스 정의와 텐서 명세를 모듈과 입력 생성기로 바꾼다. 네 조합 중 H100의 특화 모델만 0.25를 기록하였고, 나머지 셋은 0에 머물렀다. 이는 Triton 버전과 연산 아키텍처의 차이를 시사한다. 어텐션과 mixture-of-expert 과제는 구조화된 입력 합성이 필요하므로 향후 과제로 남긴다.

J GPU 너머: 두 NPU 타깃

본 연구의 발견이 GPU와 Triton의 성질인지, 아니면 생성-검증 루프 자체의 성질인지 확인하기 위해 하니스의 구조를 바꾸지 않고 프로그래밍 모델이 다른 두 NPU로 이식하였다. 첫 번째 타깃은 Triton 방언을 제공하므로 모델에게 익숙한 언어이다. 두 번째 타깃은 어떤 모델도 대규모로 본 적이 없는 Rust 내장 도메인 특화 언어를 제공한다. 두 실험 모두 실패 규칙 메모리 연구와 같은 A/B 설계를 쓴다. 조건 A는 벤더 문서만 주고, 조건 B는 장치에서 실측한 제약 한 쪽을 추가한다.

J.1 상용 NPU의 Triton 방언

첫 번째 타깃은 벤더 컴파일러를 통해 Triton 커널을 받는 데이터센터용 NPU이다. 트랜스포머 블록에 필요한 연산자 다섯 개, 즉 GELU, 계층 정규화, 행렬 곱, 어텐션, 블록 스케일 곱을 골라 현재 게이트웨이의 모델 여섯 개에 각 3개 샘플을 요청하였고, 조건당 90개 후보를 얻었다. 모든 후보는 장치 호스트에서 컴파일 게이트를 거친 뒤 NPU에서 PyTorch 참조와 비교하여 실행되며, 정확성 척도는 신호 대 양자화 잡음비(SQNR), 지연시간은 3회 실행의 중앙값이다.

표 13이 결과이다. 조건 A에서는 90개 후보 중 85개가 컴파일되지만 정답은 66개뿐이며, 그 격차의 원인은 하나로 집중된다. 실패 24건 중 16건은 컴파일되고 실행되지만 초기화되지 않은 메모리를 반환하는 커널이다. 벤더의 지원 연산 목록에는 시그모이드, 오차 함수, 최솟값, 조건 선택이 들어 있지만 어느 것도 장치에서 lowering되지 않으며, 텐서 식 안의 파이썬 스칼라 리터럴도 lowering되지 않는다. lowering이 실패하면 런타임은 조용히 폴백하고 출력 버퍼는 끝내 쓰이지 않는다. GELU는 정확히 이 연산들을 필요로 하므로 모든 모델에서 GELU 후보 18개가 전부 컴파일되고 전부 틀렸다. 컴파일 게이트는 이 실패를 보지 못하며, 컴파일된 커널 수를 세는 어떤 보고도 마찬가지이다. 수치 비교를 동반한 실행만이 이를 잡아내며, 이는 Finding 2의 검증 강도 축이 두 번째 플랫폼에서 다시 나타난 것이다.

Table 13: Triton 방언 NPU에서 15개(연산자 5개 × 샘플 3개) 중 정답 커널 수. 조건 A는 벤더 문서, 조건 B는 실측 제약 한 쪽을 추가한 것이다. 실패한 커널도 모두 컴파일되었으므로 수리 라운드는 한 번도 발동하지 않았다.

모델	A	B
claude-opus-4-8	12/15	15/15
claude-sonnet-5	12/15	15/15
gpt-5.5	12/15	15/15
gemini-3.1-pro-preview	12/15	15/15
deepseek-v4-pro	10/15	15/15
qwen3.8-max	8/15	13/15
전체	66/90	88/90
그중 GELU	0/18	18/18

조건 B는 실제로 lowering되는 연산, 텐서로 상수를 만드는 법, 지수 함수로 시그모이드를 구성하는 법을 적은 한 쪽을 추가한다. 이 한 쪽으로 플래그십 모델 다섯 개는 15개 중 15개에 도달하고 전체는 90개 중 66개에서 88개로 오른다. 남은 실패 두 건은 게이트웨이 타임아웃이다. 생성된 커널은 손으로 작성한 참조와 같은 지연시간으로 실행되며, 두 조건 모두 지연시간 비율 중앙값은 0.99이다. 같은 설정의 로컬 8B 오픈 모델은 시도한 세 연산자 중 행렬 곱 하나만 해결하였으므로, 위 결과는 플래그십 세대의 성질이다. 즉 실측 플랫폼 지식은 모델이 이미 그 언어를 유창하게 쓸 때 완전히 전이되며, 움직인 수치는 컴파일률이 아니라 컴파일된 커널 중 정답의 비율이었다.

J.2 두 번째 NPU의 Rust 내장 DSL

두 번째 타깃은 Triton과 아무것도 공유하지 않는다. 공개 커널 최적화 챌린지에서 가져온, 다른 NPU용 Rust 내장 도메인 특화 언어로 작성된 12B 디코더의 어텐션 · 피드포워드 커널 세 개이다. 하니스는 허용 파일 밖의 편집이나 장치 함수 시그니처 변경을 거부하는 계약 게이트를 추가하고, 컴파일 게이트는 결과를 Rust 프런트엔드 오류, 백엔드 오류, 정적 스케줄 길이를 가진 컴

Table 14: Rust 내장 DSL로 작성된 NPU 커널 세 개에 대한 30B 오픈 코더 모델의 결과, 작은 예산. A는 벤더 문서, B는 실측 제약을 추가한 것이다. 후보는 컴파일되고 정적 스케줄을 바꾸며 그 길이를 줄일 때에만 개선으로 센다. 마지막 열은 계약 위반 수이다.

조건	컴파일	No-op	개선	위반
A	7/9	7	0	1
B	7/9	6	0	3

파일 성공 커널로 분류한다. 컴파일되는 커널이 개선된 커널은 아니므로 컴파일 게이트 뒤에 no-op 탐지기 두 개를 둔다. 주석을 제거한 소스의 동일성과, 소스 위치를 제거한 정적 스케줄의 동일성이다. 30B 오픈 코더 모델을 커널당 샘플 3개, 수리 라운드 3회로 두 조건 모두에서 실행하였다.

표 14은 42회 시도와 약 400K 출력 토큰에 대한 결과이다. 18개 샘플 중 14개가 컴파일되었고 그중 13개는 no-op이었다. 모델은 주석을 추가하거나, 보조 함수의 이름을 바꾸거나, 컴파일러를 만족시키기 위해 수리 라운드에서 자신의 편집을 되돌렸다. 스케줄을 바꾼 유일한 후보도 그 길이는 그대로였다. 전체 시도 중 Rust 프런트엔드가 57%를 거부했고, 그 거부의 약 5분의 4는 타입 위치에 매핑 매크로를 쓴 하나의 문법 규칙이었다. 백엔드 컴파일러에 도달한 후보는 없었다. 여기서는 실측 제약이 도움이 되지 않았다. 문서는 무엇을 해야 하는지 설명하지만 모델은 그것을 이 언어로 표현하지 못했고, 계약 위반은 1건에서 3건으로 늘었다. 같은 과제의 수작업 최적화 커널은 기준선보다 약 7배 빠르게 실행되며, 모델은 그 격차를 전혀 회수하지 못했다.

두 타깃을 합치면 Finding 8이 더 선명해진다. 같은 실측 지식 한 쪽이 모델이 이미 그 언어를 쓰는 곳에서는 정답률을 90개 중 66개에서 88개로 옮겼고, 그렇지 않은 곳에서는 아무것도 옮기지 못했다. 예산과 지식은 문법의 벽 뒤에서 작용하지 그 앞에서 작용하지 않으며, 타깃이 어느 단계에서 실패하는지가 추가 투자로 무엇을 살 수 있는지를 결정한다.

K 교차 하드웨어 연구 상세

동일 조건에서 30B 오픈 모델은 Level 1의 처음 20개 과제에서 96GB GPU에서 0.75, H100에서 0.60을 기록하였다. 주원인은 GPU 메모리이다. H100은 80GB이므로 30B 모델을 서빙한 후 채점기에는 약 43GB만 남은 반면, 96GB GPU에는 약 52GB가 남았다. 입력이 큰 과제에서는 이 차이만으로 메모리 부족 실패 여부가 결정된다. 메모리 부족 사례를 보정하면 H100 수치는 약 0.70으로 올라 96GB GPU에 근접한다. 서빙 비중을 더 낮추는 것도 시도했지만, key-value 캐시가 너무 작아져 서버가 아예 시작되지 않았으므로 이 환경에는 하한이 존재한다.

속도 향상은 반대로 하드웨어에 강하게 의존한다. 정사각 행렬 곱셈에서 96GB GPU는 1.61배, H100은 0.83배를 기록하였다. H100의 cuBLAS 기준선이 매우 강력하므로 생성된 Triton 커널은 그곳에서 불리하다. 속도 향상 중앙값은 1.51 대 1.44, Fast-2 비중은 0.10 대 0.05였다.

48GB의 세 번째 GPU에서 실행을 반복한 결과, Level 1 0.37, Level 2 0.12, Level 3 0.04로 Level 전반에 걸쳐 유사한 양상을 얻었다. 이는 메모리가 충분하면 정확도가 GPU에 거의 독립적이라는 결론을 뒷받침한다. 그러나 이 GPU의 처리량은 H100의 약 4분의 1에서 5분의 1에 불과하여, 같은 스윕에 벽시계 시간으로 약 세 배가 더 걸렸다. 정확도는 하드웨어에 독립적이지만, 실험 수행 가능성 자체는 그렇지 않다.

L 재현을 위한 실무 노트

여러 운영 요인이 측정을 한 번 이상 왜곡하였다. 이들은 놓치기 쉽고 정확도 수치만으로는 탐지하기 어렵기 때문에 여기에 나열한다.

첫째, 서빙과 채점이 같은 메모리를 두고 경쟁해서는 안 된다. 서빙 프로세스가 GPU를 지나치게 많이 차지하면, 발견 2가 보여 주듯 사소한 커널조차 실패한다. 둘째, 공개된 모델 산출물은 자체 생성 한도를 가질 수 있으며 서버가 클라이언트 한도를 그 값으로 제한하므로, 클라이언트 측 한도만 올려서는 잘림이 해소되지 않는다. 셋째, 추론 모델은 코드를 작성하기 전에 수천 토큰을 소비하므로, 에이전트 프레임워크의 일반적인 응답 한도는 너무 작아 코드 블록 이전에 답을 잘라 버린다. 넷째, 서빙 컨텍스트 길이는 코드와 에러 텍스트에 따라 커지는 수리 프롬프트를 수용해야 하며, 그렇지 않으면 모든 수리 라운드가 소실된다. 다섯째, 하니스가 아직 고장 나 있던 동안 채워진 경험 메모리는 이후 실행을 계속 오염시키므로, 인프라 수정 후에는 저장소를 초기화해야 한다. 여섯째, 공유 저장소는 그 피해를 서버 간에 전파하므로, 각 실험은 자체 경로를 사용해야 한다.

과제 수준의 병렬화가 해로울 수 있다는 점도 확인하였다. 두 과제를 동시에 실행하자 서빙 슬롯이 포화되고, 슬롯당 처리량이 떨어졌으며, 생성이 클라이언트 타임아웃을 초과하고, 재시도 루프가 종료되지 않아 어떤 과제도 완료되지 못했다. 본 하드웨어에서는 순차 실행이 더 나은 선택이었다.

Table 15: 소규모 예산에서 8B 오픈 모델의 Level 1 전체 100개 과제 반복 실행. 첫 번째 행은 본문의 하니스(실행: 0.47, 0.44, 0.39, 0.47), 두 번째 행은 프레임워크 자체의 명령줄 경로(실행: 0.31, 0.41, 0.33)를 사용한다. 합집합과 교집합은 각 행에서 어느 한 실행이 해결한 과제 수와 모든 실행이 해결한 과제 수를 센다.

경로	실행 수	평균 ± 표준편차	합집합	교집합
본문 하니스	4	0.44 ± 0.04	59	25
명령줄 경로	3	0.35 ± 0.05	50	23

M 실행 간 분산과 하니스 민감도

본문의 모든 수치는 단일 실행이며, 예외는 소규모 예산에서의 8B 오픈 모델 Level 1 정확도로, 이 벤치마크의 한 번의 실행이 얼마나 변동하는지 측정하기 위해 반복하였다. 표 15에 실행 결과를 나열한다. 샘플링 temperature는 0.2이므로, 분산은 주어진 커널에 대해 결정론적인 채점기가 아니라 생성과 수리 과정의 샘플링에서 비롯된다.

두 가지 관찰이 따른다. 첫째, 단일 전체 스윕의 표준편차는 4–5포인트이다. 본 논문의 표에서 두 행 사이에 그 정도 크기의 차이는 잡음 범위 안에 있으며, 10포인트 이상인 모델 세대 간 격차만이 안전하게 해석 가능하다. 둘째, 해결된 과제의 집합은 개수보다 더 크게 변동한다. 본문 하니스의 네 실행은 25개 과제에서만 일치하지만, 함께 합치면 59개를 포괄하며, 이는 최고 단일 실행보다 12개 많다. 이는 발견 4가 수리 깊이에 대해 보고한 것과 같은 효과이며, 여기서는 예산 변경 없이 나타난다. 독립 실행 형태의 폭(breadth)은 깊이(depth)가 얻지 못하는 포괄 범위를 확보한다.

두 번째 블록은 모델 효과가 아니라 하니스 효과이다. 두 경로 모두 같은 모델, 같은 프롬프트 템플릿, 같은 과제 집합을 샘플 1개와 수리 라운드 3회로 로드한다. 차이는 다섯 가지 설정에 있다. 본문 하니스는 과제당 운영체제 프로세스 하나를 순차 실행하고, 과제당 최대 2400초를 기다리며, temperature 0.2로 샘플링하고, 두 개의 무작위 입력으로 검증하며, 프로파일링을 끈다. 명령줄 경로는 두 과제를 병렬 스레드로 실행하고, 각 평가를 120초 후 중단하며, temperature 0.3으로 샘플링하고, 다섯 개의 무작위 입력으로 검증하며, 프로파일러를 실행한다. 이 설정들 중 어느 것도 모델의 일부가 아니지만, 함께 평균을 9포인트 이동시키며, 이는 실행 간 표준편차의 두 배 이상이다. 이것이 모든 수치와 함께 3절의 설정을 보고하는 이유이며, 발견 2가 이 설정들을 정확도 보고에 명시해야 할 축에 포함하는 이유이다.

N 숨은 변수와 재현 체크리스트

측정을 한 번 이상 왜곡한 요인들을 나열한다. 이들에게는 공통점이 하나 있다. 어느 것도 정확도 보고에 나타나지 않는다는 점이다.

- **서빙 메모리와 채점 메모리를 분리한다.** 서빙 프로세스가 GPU를 지나치게 많이 차지하면 사소한 커널조차 메모리 부족으로 실패한다. 서빙과 채점이 하나의 GPU를 공유할 때에는 서빙 비중을 채점기의 필요량에 맞춰야 한다.
- **모델 산출물 내부의 생성 한도를 확인한다.** 공개된 generation config의 최대 생성 길이는 서버에 의해 제한된다. 클라이언트 측 한도만 올려서는 잘림이 해소되지 않으므로, 산출물 자체를 확인해야 한다.
- **프레임워크의 응답 토큰 한도를 높인다.** 추론 모델은 코드를 작성하기 전에 사고 과정에 수천 토큰을 소비한다. 일반적인 8K 한도는 코드 블록 이전에 답을 잘라 해결 가능한 과제를 코드 없는 과제로 만든다. 실험에서 한도를 13,000으로 높이자 영향을 받은 범위의 정확도가 0.37에서 0.78로 상승하였다.
- **서빙 컨텍스트를 충분히 길게 한다.** 수리 프롬프트는 코드와 에러 텍스트에 따라 커진다. 컨텍스트가 짧으면 모든 수리 라운드가 소실된다.
- **인프라 수정 후 경험 메모리를 초기화한다.** 하니스가 고장 난 동안 수집된 규칙은 정상 환경에서도 정확도를 계속 떨어뜨린다.
- **각 실험에 고유한 저장소 경로를 부여한다.** 여러 서버가 하나의 경험 저장소를 공유하면 오염이 한 실험에서 다음 실험으로 전파된다.
- **소규모 서빙 클러스터에서는 순차 실행을 선호한다.** 서빙 슬롯이 포화되면 처리량이 떨어지고 재시도 루프가 종료되지 않는다.

마지막으로, 모든 정확도 수치 옆에 보고해야 할 축을 다시 정리한다. 시도 예산, 검증 강도, 채점기의 메모리 여유분, 서빙 컨텍스트 길이, 프레임워크와 모델 산출물 양쪽의 응답 토큰 상한, 평가 분포가 그것이다. 이러한 축 없이 수치를 비교하는 것은 서로 다른 체제에서 얻은 측정값을 비교하는 것이다.